

Roteiro do vídeo: bcrypt em funcionamento

Duração: cerca de 3 a 4 minutos. Gravação de tela do terminal. Paulo Victor: abertura e Partes 1 a 3. Isabela: Partes 4 e 5, menu e encerramento. Os valores de sal e hash mudam a cada execução; falem 'por exemplo' e leiam o que aparecer na tela.

Antes de gravar: abra o terminal na pasta, rode `python bcrypt_seminario.py` uma vez para testar (leva uns 15 s até o menu) e deixe a janela grande, com fonte legível.

Abertura (Paulo, 0:20)

NA TELA: terminal vazio, na pasta do projeto; digita `python bcrypt_seminario.py`

Olá, somos o Paulo Victor e a Isabela, do tema 9, bcrypt. Vamos rodar a nossa implementação do bcrypt, escrita do zero em Python, sem nenhum import, e explicar o que aparece na tela.

Parte 1 - Blowfish e os dígitos de pi (Paulo, 0:35)

NA TELA: PARTE 1: `P[0] = 243F6A88`, as três rodadas e o Resultado

O bcrypt é construído sobre a cifra Blowfish, que começa com números tirados dos dígitos de pi. Nós calculamos pi no próprio código, e aqui o programa confere: `P[0]` é `243F6A88` e `S0[0]` é `D1310BA6`, os mesmos valores do artigo original do Blowfish.

Abaixo vemos as três primeiras das 16 rodadas de uma rede de Feistel. Em cada rodada, a função F mistura uma metade do bloco e as duas metades trocam de lugar. É essa cifragem de bloco que o bcrypt repete milhares de vezes.

Parte 2 - bcrypt passo a passo (Paulo, 0:50)

NA TELA: PARTE 2: senha 'abc', custo 4, os Passos 1 a 6, e a contagem de cifragens

Agora o bcrypt com a senha 'abc' e custo 4, o que significa 2 elevado a 4, ou 16 repetições. O sal são 16 bytes aleatórios; cada execução gera um diferente.

Passo 1: a senha vira 4 bytes: 'abc' mais um byte nulo. Passo 2: o estado começa com os dígitos de pi, `P[0]` igual a `243F6A88`. Passo 3: misturamos o sal e a senha, e o `P[0]` muda para outro valor. Passo 4: repetimos as 16 voltas e o `P[0]` muda de novo. Esse laço é a parte lenta de propósito. Passo 5: cifra-se 64 vezes um texto fixo, e dos 24 bytes ficamos com 23. Passo 6: montamos o hash final.

Embaixo, o programa mostra 17.385 cifragens, igual ao esperado pela fórmula. E a anatomia do hash: versão \$2b\$, custo 04, o sal com 22 caracteres e o hash com 31, 60 caracteres no total.

Parte 3 - Vetores oficiais (Paulo, 0:25)

NA TELA: PARTE 3: três linhas com OK

Para provar que a implementação está correta, usamos três exemplos gerados pelo bcrypt oficial, entre eles o clássico 'U*U'. Com o mesmo sal e a mesma senha, o nosso código produz exatamente o mesmo hash. Os três deram OK.

Parte 4 - Sal, login e ataque (Isabela, 0:55)

NA TELA: PARTE 4: hash 1 e hash 2, Login certo/errado, tentativas do ataque

Aqui geramos dois hashes da mesma senha, 'senha123', e eles são diferentes, porque

cada um tem um sal aleatório. Isso impede tabelas pré-calculadas e esconde senhas repetidas no banco.

No login, a senha certa retorna True e a errada, False: o sistema não desfaz o hash, apenas recalcula com o sal guardado e compara.

Depois simulamos um atacante que roubou o hash e tenta uma lista de senhas comuns. Ele acerta na sexta tentativa, 'senha123', gastando 104 mil cifragens. Ou seja: o bcrypt não salva uma senha fraca, ele só deixa cada palpite mais caro.

Parte 5 - Limite de 72 bytes (Isabela, 0:25)

NA TELA: PARTE 5: Hashes iguais? True

Uma limitação do bcrypt: só os primeiros 72 bytes da senha contam. Aqui duas senhas diferentes só depois do byte 72 geram o mesmo hash. Por isso, em sistemas reais, senhas longas passam por um pré-hash.

Menu e encerramento (Isabela, 0:40)

NA TELA: MENU: digitar 1, senha qualquer (ex.: videoteste), custo 4; depois 0 para sair

Por fim, o menu interativo. Na opção 1 podemos gerar o hash de qualquer senha, vendo os passos. Aqui digitamos uma senha e o custo 4, e o programa mostra o hash completo.

Resumindo: o bcrypt usa sal aleatório e um custo ajustável para tornar cada tentativa de adivinhar uma senha muito mais cara, e a nossa versão em Python puro bate com os vetores oficiais. Obrigada por assistir!

Dicas de gravação

- Pause a gravação (ou aguarde) enquanto o programa roda; o trecho das Partes 1 a 5 leva uns 15 segundos. Rolar o terminal para cima ajuda a mostrar cada parte enquanto se fala.
- Use custo 4 ou 5 no menu. Custos maiores demoram muito em Python puro.
- Se esquecerem algum valor, digam 'o valor muda a cada execução' e apontem para a tela.
- Entreguem o vídeo no Classroom junto com o código, até as 19h do dia da apresentação.